Experiment 4

Aim: Write a program to remove Left Recursion from a Grammar

Theory:

A grammar is said to have left recursion when a non-terminal A has a production of the form A → Aα, meaning the derivation of A begins by referring back to A itself. This is a problem for top-down parsers (like recursive-descent or LL parsers), because trying to expand A leads straight back to expanding A again, causing infinite recursion without ever consuming any input. Left recursion is removed by rewriting productions of the form A → $A\alpha_1$ | $A\alpha_2$ | ... | β_1 | β_2 | ... (where the β's don't start with A) into a right-recursive equivalent: A → β_1A' | β_2A' | ... and A' → α_1A' | α_2A' | ... | ε, where A' is a newly introduced non-terminal and ε denotes the empty string. This transformation preserves the language generated by the grammar while eliminating the immediate left recursion, making the grammar suitable for predictive parsing techniques used in the syntax-analysis phase of a compiler.

Code:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;


vector<string> splitAlternatives(const string &rhs) {
   vector<string> parts;
   string current = "";
   for (char c : rhs) {
      if (c == '|') {
         parts.push_back(current);
         current = "";
      } else {
         current += c;
      }
   }
```

```
    parts.push_back(current);
    return parts;
}

void removeLeftRecursion(const string &production) {
    int arrowPos = production.find("->");
    string lhs = production.substr(0, arrowPos);
    string rhs = production.substr(arrowPos + 2);

    vector<string> alternatives = splitAlternatives(rhs);
    vector<string> alpha, beta;

    for (const string &alt : alternatives) {
        if (alt.substr(0, lhs.length()) == lhs) {
            alpha.push_back(alt.substr(lhs.length()));
        } else {
            beta.push_back(alt);
        }
    }

    if (alpha.empty()) {
        cout << production << "  (no left recursion)\n";
        return;
    }

    string newNonTerminal = lhs + "'";

    cout << lhs << " -> ";
```

```
    for (size_t i = 0; i < beta.size(); i++) {
        cout << beta[i] << newNonTerminal;
        if (i != beta.size() - 1) cout << " | ";
    }
    cout << "\n";

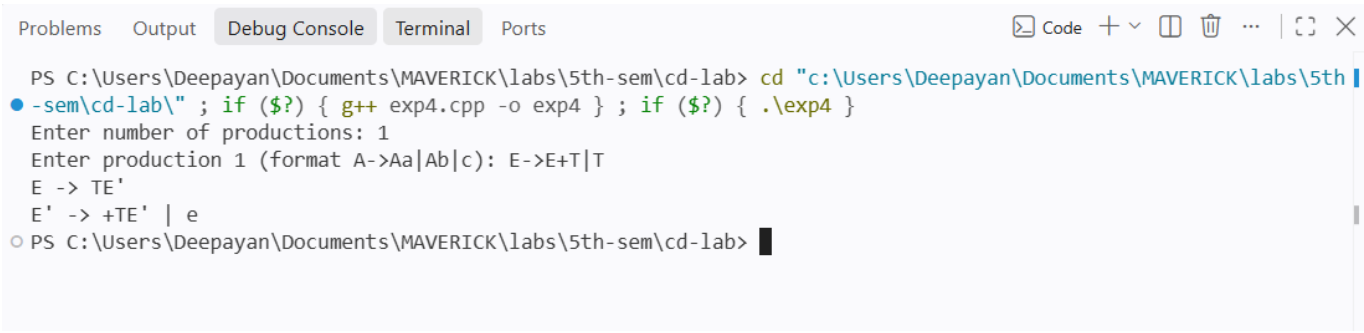
    cout << newNonTerminal << " -> ";
    for (size_t i = 0; i < alpha.size(); i++) {
        cout << alpha[i] << newNonTerminal << " | ";
    }
    cout << "e\n";
}

int main() {
    int n;
    cout << "Enter number of productions: ";
    cin >> n;
    cin.ignore();

    for (int i = 0; i < n; i++) {
        string production;
        cout << "Enter production " << (i + 1) << " (format A->Aa|Ab|c): ";
        getline(cin, production);
        removeLeftRecursion(production);
    }

    return 0;
}
```

Output:


Problems
Output
Debug Console
Terminal
Ports
Code
PS C:\Users\Deepayan\Documents\MAVERICK\labs\5th-sem\cd-lab> cd "c:\Users\Deepayan\Documents\MAVERICK\labs\5th-sem\cd-lab\" ; if ($?) { g++ exp4.cpp -o exp4 } ; if ($?) { .\exp4 }
Enter number of productions: 1
Enter production 1 (format A->Aa|Ab|c): E->E+T|T
E -> TE'
E' -> +TE' | e
PS C:\Users\Deepayan\Documents\MAVERICK\labs\5th-sem\cd-lab>